

applications, infrastructure and data. They extract the maximum out of IT investments by advocating component reuse, and object-based development and maintenance.

Reviewer: As reviewers, SAs review project architectures for compliance to standards. They focus on:

- Executing and participating in project review and approval processes at designated and ad-hoc review points via the EA Review Board (EARB). This includes participation in hardware/software evaluation and selection processes.
- Validating architecture designs and giving recommendations.

Enforcer: SAs enforce compliance to architecture standards. They focus on defining and governing project review and approval processes, which include explicit architecture review and sign-off tollgates, and active participation in hardware and software purchases. They decide on issues involving non-compliant projects and review overall costs as well as the risks associated with exceptions.

Subject matter expert: SAs need to have deep knowledge in one or more domains, and provide advice on architecture development related to their area of expertise.

Solution architect: SAs play an active role in architecting solutions for projects. They help to architect solutions that embody compliance and component reuse, and look at technology innovation to address project requirements. They consider available standards where applicable but are not constrained by them.

Common mistakes software architects make

Here is a list of some of the worst architecture practices we have come across in our experience so far:

- No strategic planning and budgeting
- Following software architecture frameworks strictly
- Excessive standardisation
- Analysis paralysis
- Lack of business focus and a common direction
- Focusing on the language rather than the outcome
- Ivory tower approach; working in silos
- Non-adoption of next-generation technologies
- Lack of integration, compatibility and interoperability between applications
- Complex, fragile and costly interfaces between incongruent applications

- Lack of open communication
- Lack of a feedback loop
- Weak technical talent and skillset
- Weak architecture governance; no metrics
- No central repository of architecture repository
- Architecture driven by tools
- Focusing primarily on the current state

Here's a list of the most common mistakes SAs make and what they should do to avoid them.

Over engineering: SAs must avoid unnecessary complexity, keeping solutions as simple as possible.

Ignore the bigger picture: They must avoid focusing too narrowly on current tasks and keep the overall architecture in mind.

Neglect technical debt: Accumulation of technical debt must not be ignored and addressed promptly to maintain code quality and system performance. Attention must be paid to non-functional requirements like security, performance, availability, scalability, and maintainability.

Isolate themselves: SAs should not work in isolation. Collaboration and communication are key.

Overlook documentation updates: They mustn't forget to regularly update documentation as the architecture evolves.

Underestimate the importance of communication: Clear and constant communication with the team and stakeholders is necessary and critical.

Ignore feedback: SAs must listen to feedback from developers and stakeholders to improve the architecture.

Over-reliance on one technology: They should not be overly dependent on a single technology or vendor and consider diversifying their tech stack.

Avoid automation: They must not neglect the power of automation in testing, deployment, and monitoring applications, and ensure thorough testing is part of the development lifecycle.

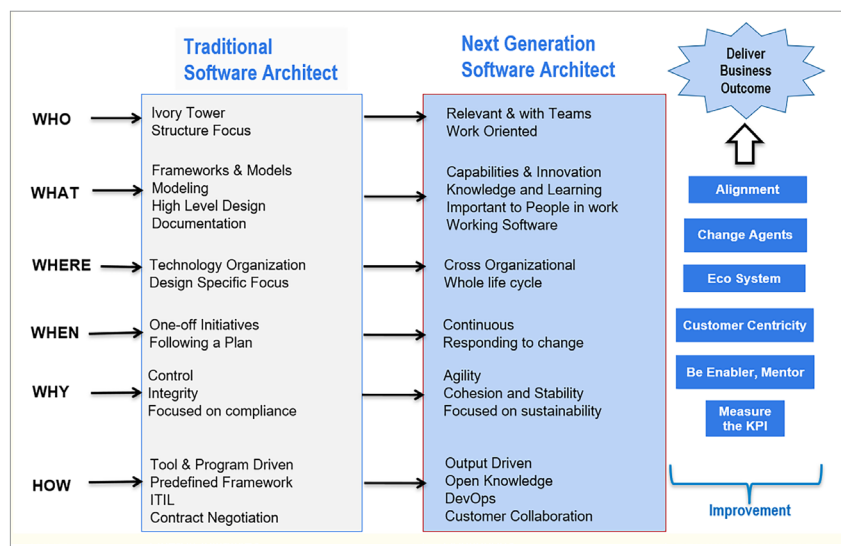


Figure 2: Shaping the next generation software architect